

HabitFlow API Specification

Team-404

Version 1.1

Last Updated: 2026, Sept. 30

Prepared by: Renz Mathieu M. Raquin (Dashboard API Spec)

Endpoints marked with an Authorization parameter require the header Authorization: Bearer <token> from /api/auth/login. Every authenticated request only sees data owned by that user.

VI. Task Dashboard (api/dashboard)

The Task Dashboard shows the user's task counts and lets them view their tasks across all projects with sort, filter, group by and layout options (list, board and gantt).

A. View Task Dashboard

API Endpoint	/api/dashboard	
method	GET	
Request Parameters	Type	Description
Authorization	string	Bearer token of the authenticated user.
<i>Returns a compact task summary for the authenticated user. Counts follow the Scheduling Rules in IV.A. The task lists behind each count are loaded with List Dashboard Tasks (VI.C), e.g. statuses=current for Today, upcoming for Upcoming and overdue for Overdue.</i>		
Success		
Status Code	200	
Response Type	json	
Response Data	Type	Description
currentCount	integer	Current task count.
upcomingCount	integer	Upcoming task count.
overdueCount	integer	Overdue task count.
completedCount	integer	Completed task count.
Sample Response		
{ "currentCount": 2, "upcomingCount": 5, "overdueCount": 1, "completedCount": 12 }		
Fail		
Status Code	401	
Response Type	json	
Response Data	Type	Description
error	string	Authentication error.
Sample Response		
{ "error": "Unauthorized" }		

B. View Options

View options control how the Dashboard shows tasks: which tasks appear (filter), their order (sort), how they are grouped (group by) and the layout (list, board or gantt). They are sent as query parameters to List Dashboard Tasks (VI.C) and are not stored in the database, so the frontend keeps the user's current choice. Every option is based on an existing `tbl_task` column, a linked table (`tbl_project`, `tbl_priority_level`, `tbl_label` through `tbl_task_label`) or the computed schedule status (IV.A).

Layouts (layout)

layout	Description
list	Default. Tasks are shown as a list, one section per group.
board	Kanban board. Each group is a column and each task is a card. If <code>groupBy</code> is none, the board groups by status. Moving a card to another column calls the matching Task endpoint (e.g. Move Task to Project III.E, Update Task Priority III.H, Set Task Labels III.I, Update Task Due Date III.J, Complete / Reopen Task IV.D).
gantt	Gantt chart (timeline). Each task is a bar from its <code>dueDate</code> to its deadline, or a single day if it has no deadline. Groups are shown as rows (swimlanes). Tasks without a <code>dueDate</code> are not shown.

Sort fields (sortBy)

sortBy	Column	Sorts tasks by
manual	<code>tbl_task.sort_order</code>	Display order set with Update Task Sort Order (III.M). Default.
title	<code>tbl_task.title</code>	Task title, alphabetically.
priority	<code>tbl_task.priority_id</code>	Priority (ASC shows Priority 1 first).
dueDate	<code>tbl_task.due_date</code> , <code>tbl_task.due_time</code>	Due date, then due time. Tasks without a due date are always last.
deadline	<code>tbl_task.deadline</code>	Deadline. Tasks without a deadline are always last.
createdAt	<code>tbl_task.created_at</code>	Date the task was added.

`sortDirection` is ASC or DESC (default ASC). Tasks with the same value are ordered by `sort_order`. Sorting applies to the tasks inside each group.

Group fields (groupBy)

groupBy	Column	Groups tasks by
none	-	No grouping. All tasks are returned in one group. Default.
project	<code>tbl_task.project_id</code>	Project. Groups follow the projects' <code>sort_order</code> .
priority	<code>tbl_task.priority_id</code>	Priority level. Groups follow <code>tbl_priority_level.sort_order</code> .
label	<code>tbl_task_label</code>	Label, by label name. A task with several labels appears in each of its label groups. Tasks without labels go to a No label group.

dueDate	tbl_task.due_date	Due date. One group per date, plus a No date group.
status	computed (IV.A)	Schedule status, in the order overdue, current, upcoming, unscheduled, completed.

groupDirection is ASC or DESC (default ASC) and orders the groups themselves. DESC reverses the order listed above. Empty groups are left out, except on a board grouped by priority or status, where every column is shown.

Filter fields

Filter	Column	Shows only tasks that
projectIds	tbl_task.project_id	Belong to one of these projects.
priorityIds	tbl_task.priority_id	Have one of these priorities.
labelIds	tbl_task_label.label_id	Have at least one of these labels.
statuses	computed (IV.A)	Have one of these schedule statuses.
dueFrom	tbl_task.due_date	Have a due date on or after this date.
dueTo	tbl_task.due_date	Have a due date on or before this date.

- List filters are comma-separated (e.g. statuses=current,overdue). Values inside one filter are combined with OR, and different filters are combined with AND.
- A filter that is not sent is not used. With no filters, every task of the user is shown.
- Deleted tasks and tasks of deleted projects (deleted_at set) are never shown.

C. List Dashboard Tasks

API Endpoint	/api/dashboard/tasks	
method	GET	
Request Parameters	Type	Description
Authorization	string	Bearer token of the authenticated user.
layout	string	Optional. list, board or gantt (default is list).
sortBy	string	Optional. manual, title, priority, dueDate, deadline or createdAt (default is manual).
sortDirection	string	Optional. ASC or DESC (default is ASC).
groupBy	string	Optional. none, project, priority, label, dueDate or status (default is none; status on a board).
groupDirection	string	Optional. ASC or DESC (default is ASC).
projectIds	string	Optional. Comma-separated project IDs.
priorityIds	string	Optional. Comma-separated priority IDs.
labelIds	string	Optional. Comma-separated label IDs.
statuses	string	Optional. Comma-separated schedule statuses (see IV.A).
dueFrom	string	Optional. Earliest due date (YYYY-MM-DD).

dueTo	string	Optional. Latest due date (YYYY-MM-DD). Must not be earlier than dueFrom.
<i>Returns the authenticated user's tasks filtered, sorted and grouped by the view options in VI.B. All parameters are query parameters and can be combined. Use projectIds with one project to show a single project as a list, board or gantt chart.</i>		
Success		
Status Code	200	
Response Type	json	
Response Data	Type	Description
view	object	Options used for this response, with defaults filled in.
view.layout	string	list, board or gantt.
view.sortBy	string	Sort field.
view.sortDirection	string	ASC or DESC.
view.groupBy	string	Group field.
view.groupDirection	string	ASC or DESC.
totalCount	integer	Number of tasks that match the filters.
groups	array	Groups in groupDirection order.
groups[].key	string	Group value (e.g. 1, overdue, 2026-10-02, none).
groups[].name	string	Display name (e.g. Priority 1, Overdue, No label).
groups[].taskCount	integer	Number of tasks in the group.
groups[].tasks	array	Tasks in sort order, with the same fields as List Tasks (III.A) plus status (IV.A).
Sample Response		
<pre>{ "view": { "layout": "board", "sortBy": "dueDate", "sortDirection": "ASC", "groupBy": "priority", "groupDirection": "ASC", "totalCount": 1, "groups": [{ "key": "1", "name": "Priority 1", "taskCount": 1, "tasks": [{ "taskId": 25, "projectId": 10, "priorityId": 1, "title": "Submit documentation", "description": null, "dueDate": "2026-10-02", "dueTime": "17:00:00", "deadline": "2026-10-02", "completedAt": null, "sortOrder": 0, "isRecurring": false, "labels": [], "status": "upcoming" }] }, { "key": "2", "name": "Priority 2", "taskCount": 0, "tasks": [] }, { "key": "3", "name": "Priority 3", "taskCount": 0, "tasks": [] }, { "key": "4", "name": "Priority 4", "taskCount": 0, "tasks": [] }] } }</pre>		
Fail		
Status Code	400	
Response Type	json	
Response Data	Type	Description
error	string	Invalid view option or date range.
Sample Response		
<pre>{ "error": "Invalid sort field" }</pre>		

Fail		
Status Code	401	
Response Type	json	
Response Data	Type	Description
error	string	Authentication error.
Sample Response		
{ "error": "Unauthorized" }		